MediVault

Patient-Controlled Medical Records Protocol on BOT Chain

WHITEPAPER | VERSION 1.0
October 2026 · BOT Chain · Chain ID 968

PROTOCOL OVERVIEW

MediVault is a patient-controlled medical records protocol deployed on BOT Chain. Admin-registered hospitals add records (as encrypted file hashes) to a patient's on-chain vault; patients grant time-limited access to doctors; doctors can read records only within the access window. No personal data or file content is stored on-chain — only IPFS CIDs or SHA-256 hashes of encrypted files. The result is a portable, patient-sovereign health record that any approved hospital or doctor can contribute to or read with the patient's consent.

A technical overview of the protocol, its design, core functions, security properties, and roadmap.

1. Problem Statement

- Siloed health data — each hospital maintains its own EHR; records are not portable across providers.
- Consent is implicit — sharing records typically requires paper forms or institutional agreements.
- Single-institution access — doctors outside the originating hospital cannot read records without fax or courier.
- Breach risk — centralised EHR databases are high-value targets; a single breach exposes millions.

2. Solution Architecture

2.1 Roles

`Role`
Permissions

Admin

Register / deactivate hospitals; transfer admin role

Hospital

Add records to any patient (by address); read records they created

`Role`
Permissions

Patient

Grant / revoke time-limited doctor access; view own records

Doctor

Read patient records during granted access window

2.2 Access Control Model

Access is governed by accessUntil[patient][doctor] — a mapping of expiry timestamps. A doctor has access if block.timestamp < accessUntil[patient][doctor]. Patients set expiry in whole days (1–365). Revocation sets the expiry to zero immediately.

3. Contract Architecture

3.1 Data Model

Hospital: name (string), active (bool).

Record: hospital address, category (string), fileHash (IPFS CID or SHA-256), note, time.

Access mapping: patient address => doctor address => expiry (uint256 unix timestamp).

3.2 Write Functions

`Function`
`Role`
Description

`addHospital(addr, name)`
Admin

Registers a hospital

`removeHospital(addr)`

Admin

Deactivates a hospital

`addRecord(patient, category, fileHash, note)`
Hospital

Appends an encrypted record hash

`grant(doctor, daysValid)`
Patient

Grants access for 1–365 days

`revoke(doctor)`
Patient

Immediately revokes doctor access

`transferAdmin(addr)`
Admin

Transfers admin role

3.3 Read Functions

Function	Returns
`getRecord(patient, id)`	Hospital, category, fileHash, note, time (requires access)
`recordCount(patient)`	Number of records for a patient
`hasAccess(patient, viewer)`	True if viewer is patient or within access window
`myDoctors()`	List of doctors and their access expiry timestamps
hospitalAt(i)	Hospital address, name, active status

4. Privacy & Security

4.1 No PHI On-Chain

Only hashes or CIDs of encrypted files are stored. Personal health information must be encrypted off-chain before the hash is submitted. This means a chain compromise exposes only meaningless hashes.

4.2 Patient Sovereignty

Patients can revoke any doctor's access at any time. Revocation is instantaneous — accessUntil is set to zero in a single transaction.

4.3 Hospital Gating

Only admin-registered hospitals can add records. This prevents arbitrary parties from polluting a patient's medical history.

4.4 Access Expiry

Doctor access is time-limited (max 365 days). This prevents forgotten grants from persisting indefinitely.

5. Events

Event	Parameters
HospitalAdded	hospital address, name
HospitalRemoved	hospital address
AccessGranted	patient, doctor, until (timestamp)
AccessRevoked	patient, doctor
RecordAdded	patient, hospital, record id

6. Deployment

Parameter	Value
Chain	BOT Chain (Chain ID 968)
RPC	https://rpc.botchain.ai
Recommended file storage	IPFS (Pinata, Web3.Storage) with client-side encryption
Encryption standard	AES-256-GCM (patient-held key) recommended

7. Limitations & Roadmap

- Not audited — use on testnet before any clinical deployment.
- Key management is the patient's responsibility — lost keys mean lost access to encrypted files.
- No HIPAA / GDPR enforcement on-chain — compliance is the responsibility of each hospital deployer.

Milestone	Description
v1.1	Emergency access pattern (break-glass with time-lock and audit trail)
v1.2	Doctor-to-doctor record forwarding with patient consent
v1.3	ERC-20 micro-payment for hospital record storage incentivisation
v2.0	ZK-proof based selective disclosure (share specific fields without revealing full record)